Name: Trần Anh Tuấn
Student ID: 24521943
Class: CS4323.Q12.CTTT

OS
LAB 3

CHECKLIST

1. Homework

	1	2	3
Explanation	☒	☒	☒
Your code	☒	☒	☒
Results and discussions	☒	☒	☒

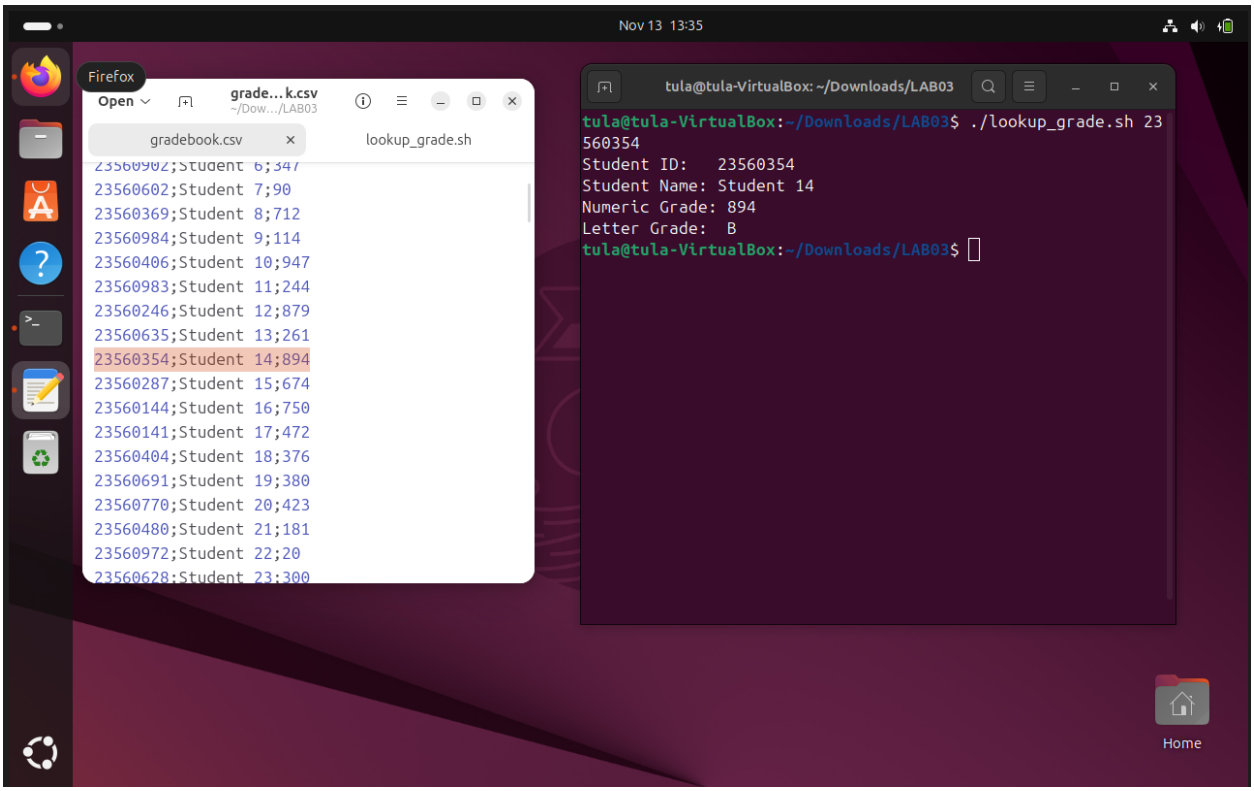
<u>Mark:</u>

2. Lab 3

1. Given the file [gradebook.csv]() which contain grades of a class. Research how to use IFS to read csv file. Use shell script to:

- **Read Student ID from command line**
- **Use Student ID to find and print name, original grade, and letter-based grade of corresponding student**

Answer:



Lookup_grade.sh

```
#!/bin/bash

# Check that a Student ID was provided
if [ $# -ne 1 ]; then
   echo "Usage: $0 <StudentID>"
   exit 1
fi

TARGET_ID=$1

while IFS=';' read -r sid name grade_raw; do
   if [ "$sid" = "StudentID" ]; then
      continue
   fi
   grade=${grade_raw//$'\r'/}
   if [ "$sid" = "$TARGET_ID" ]; then
      if [ "$grade" -ge 900 ]; then
         letter="A"
      elif [ "$grade" -ge 800 ]; then
         letter="B"
      elif [ "$grade" -ge 700 ]; then
         letter="C"
      elif [ "$grade" -ge 600 ]; then
         letter="D"
      else
         letter="F"
      fi
```

```
    echo "Student ID:   $sid"
    echo "Student Name: $name"
    echo "Numeric Grade: $grade"
    echo "Letter Grade:  $letter"
    exit 0
  fi

done < gradebook.csv

echo "Error: Student ID $TARGET_ID not found."
exit 1
```

Explanation:

1. Check Argument Count

```
if [ $# -ne 1 ]; then
    echo "Usage: $0 <StudentID>"
    exit 1
fi
```

- The script requires exactly **one argument** (Student ID).
- If the user provides too many or too few arguments, the script exits.

2. Read Student ID

```
TARGET_ID=$1
```

- Stores the supplied Student ID in `TARGET_ID`.

3. Read CSV File Line by Line

```
while IFS=';' read -r sid name grade_raw; do
```

- Uses `IFS=';'` because the CSV uses **semicolon** separators.
- Assigns values to:
  - `sid` → student ID
  - `name` → student name
  - `grade_raw` → numeric grade (may contain `\r`)

4. Remove Hidden Carriage Return Characters

```
grade=${grade_raw//$'\r'/}
```

- Fixes issues caused by Windows line endings (`\r\n`).

5. Skip Header Row

```
if [ "$sid" = "StudentID" ]; then
    continue
fi
```

- Prevents the header from being processed like a data row.

6. Match the Student ID

```
if [ "$sid" = "$TARGET_ID" ]; then
```

- When the Student ID matches, the script processes the student data.

7. Determine Letter Grade

```
if [ "$grade" -ge 900 ]; then
    letter="A"
elif [ "$grade" -ge 800 ]; then
    letter="B"
elif [ "$grade" -ge 700 ]; then
    letter="C"
elif [ "$grade" -ge 600 ]; then
    letter="D"
else
    letter="F"
fi
```

- Converts numeric grade to a **letter-based grade**.

8. Print the Student's Information

```
echo "Student Name: $name"
echo "Numeric Grade: $grade"
echo "Letter Grade: $letter"
```

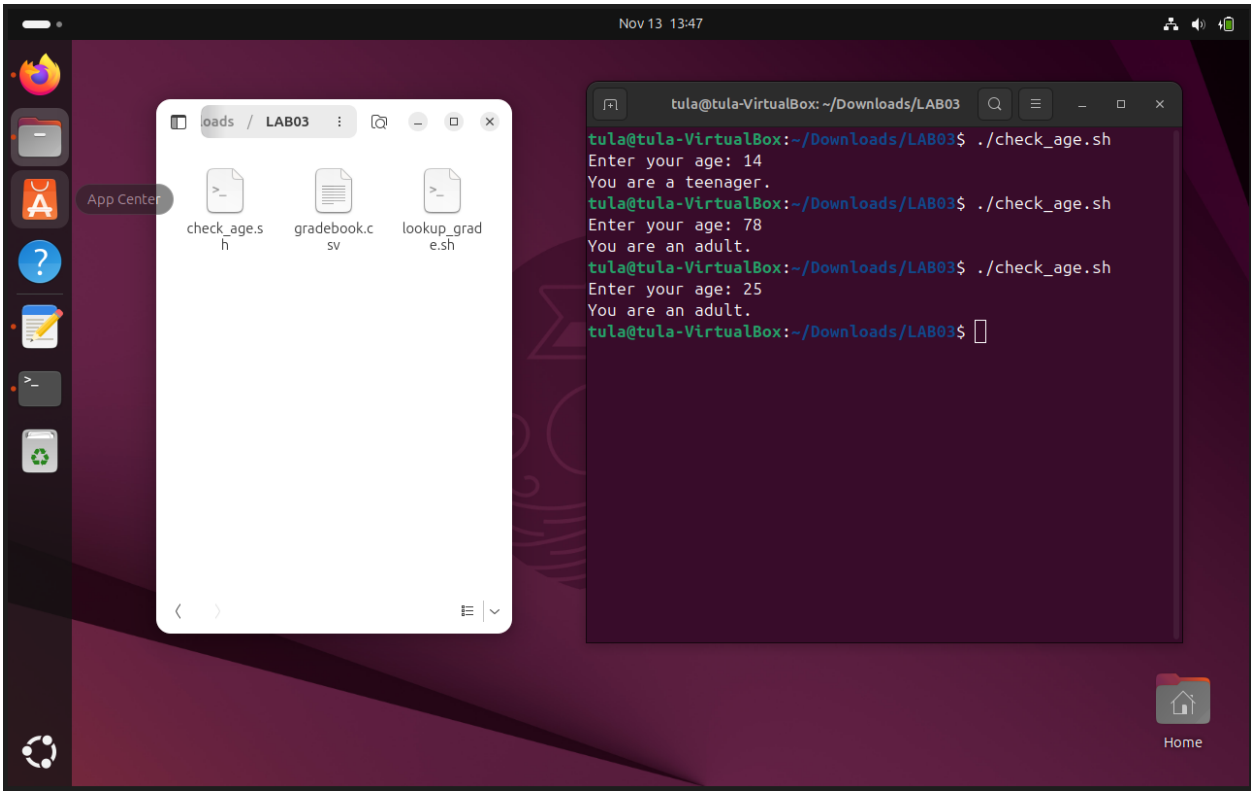
9. If No Student is Found

```
echo "Error: Student ID $TARGET_ID not found."
```

2. Write a shell script to let user input their age, then:

- **Check if input is a number and its value is between 0 and 100.**
- **If input is less than 12 → Print "You are a child"**
- **If input is from 12 – 18 → Print "You are a teenager"**
- **If input is greater than 18 → Print "You are an adult"**

Answer:



Check_age.sh

```
#!/bin/bash

read -p "Enter your age: " age

if ! [[ "$age" =~ ^[0-9]+$ ]]; then
    echo "Error: Age must be a number."
    exit 1
fi

age=$((age))
```

if ["$age" -lt 0] || ["$age" -gt 100]; **then**

echo "Error: Age must be between 0 and 100."

exit 1

fi

if ["$age" -lt 12]; **then**

echo "You are a child."

elif ["$age" -le 18]; **then**

echo "You are a teenager."

else

echo "You are an adult."

fi

Explanation:

1. Prompt for Age

```
read -p "Enter your age: " age
```

- Asks the user to type their age.
- Stores input in the variable `age`.

2. Check if Input is a Number

```
if ! [[ "$age" =~ ^[0-9]+$ ]]; then
    echo "Error: Age must be a number."
    exit 1
fi
```

- Uses a **regular expression** to verify that input contains only digits.
- If not, prints an error and ends the script.

3. Convert Input to Integer

```
age=$((age))
```

- Ensures the input is handled as a **numeric value**.

4. Check Age Range (0–100)

```
if [ "$age" -lt 0 ] || [ "$age" -gt 100 ]; then
    echo "Error: Age must be between 0 and 100."
    exit 1
fi
```

- Validates that the age is between **0 and 100**.

5. Classify the User

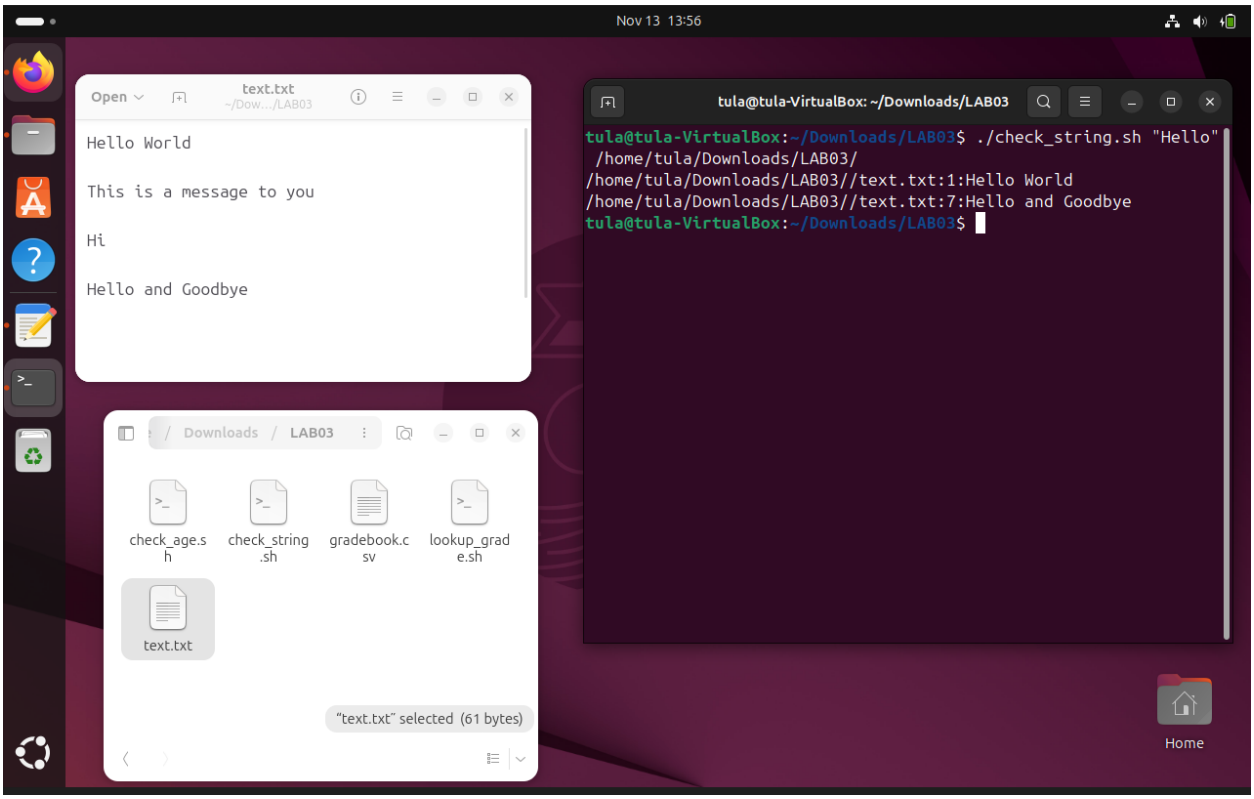
```
if [ "$age" -lt 12 ]; then
    echo "You are a child."
elif [ "$age" -le 18 ]; then
    echo "You are a teenager."
else
    echo "You are an adult."
fi
```

- Uses conditional statements to determine the age category.
- Prints the appropriate message.

3. Write a program that read 02 arguments from command line: first argument is a string while second argument is a path.

Check if that string exists in any text file (for example, test.txt) in the provided path. Print the line containing the string.

Answer:



Check_string.sh

```
#!/bin/bash

if [ $# -ne 2 ]; then
    echo "Usage: $0 <string> <path>"
    exit 1
fi

search="$1"
path="$2"

if [ ! -d "$path" ]; then
```

```
    echo "Error: '$path' is not a valid directory."
    exit 1
fi

grep -nH -R -F "$search" "$path"/*.txt 2>/dev/null

if [ $? -ne 0 ]; then
    echo "String '$search' not found in any text file in $path."
fi
```

Explanation:

1. Check Number of Arguments

```
if [ $# -ne 2 ]; then
    echo "Usage: $0 <string> <path>"
    exit 1
fi
```

- Ensures the user provides exactly **two** inputs.

2. Store Arguments

```
search="$1"
path="$2"
```

- Saves search text and target directory.

3. Check If the Path Is a Directory

```
if [ ! -d "$path" ]; then
    echo "Error: '$path' is not a valid directory."
    exit 1
fi
```

- Ensures the provided path exists and is a directory.

4. Search for the String

```
grep -nH -R -F "$search" "$path"/*.txt 2>/dev/null
```

Flag	Meaning
`-n`	Show line number
`-H`	Show filename
`-R`	Search recursively
`-F`	Treat pattern as a fixed string (not regex)

- Searches only `.txt` files.
- Hides warnings using `2>/dev/null`.

5. Check grep Result

```
if [ $? -ne 0 ]; then
    echo "String '$search' not found in any text file in $path."
fi
```

- If nothing is found, prints a message telling the user.